

Practical 2: Demonstrate various data pre-processing techniques for a given dataset.

```
import pandas as pd

from sklearn.model_selection import train_test_split

data = { 'Age': [20, 25, 30, 30, 40, 45], 'Salary': [20000, 30000,
None, 40000, 50000, 60000], 'Target': [0, 1, 1, 1, 0, 1] }

df = pd.DataFrame(data)

print("Original Dataset:")

print(df)

print("\n1. Missing Values:")

print(df.isnull().sum())

df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

print("\n2. After Handling Missing Values:")

print(df)

df = df.drop_duplicates()

print("\n3. After Removing Duplicates:")

print(df)

X = df.drop('Target', axis=1)

y = df['Target']

print("\n4. Features (X):")

print(X)

print("\nTarget (y):")

print(y)

X_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=0.2, random_state=40 )

print("\n5. Training Data:")

print(X_train)

print("\nTesting Data:")

print(X_test)
```

Original Dataset:

	Age	Salary	Target
0	20	20000.0	0
1	25	30000.0	1
2	30	NaN	1
3	30	40000.0	1
4	40	50000.0	0
5	45	60000.0	1

1. Missing Values:

Age	0
Salary	1
Target	0

dtype: int64

2. After Handling Missing Values:

	Age	Salary	Target
0	20	20000.0	0
1	25	30000.0	1
2	30	40000.0	1
3	30	40000.0	1
4	40	50000.0	0
5	45	60000.0	1

3. After Removing Duplicates:

	Age	Salary	Target
0	20	20000.0	0
1	25	30000.0	1
2	30	40000.0	1
4	40	50000.0	0
5	45	60000.0	1

4. Features (X):

	Age	Salary
0	20	20000.0
1	25	30000.0
2	30	40000.0
4	40	50000.0
5	45	60000.0

Target (y):

0	0
1	1
2	1
4	0
5	1

Name: Target, dtype: int64

5. Training Data:

	Age	Salary
0	20	20000.0
1	25	30000.0
5	45	60000.0
4	40	50000.0

Testing Data:

	Age	Salary
2	30	40000.0

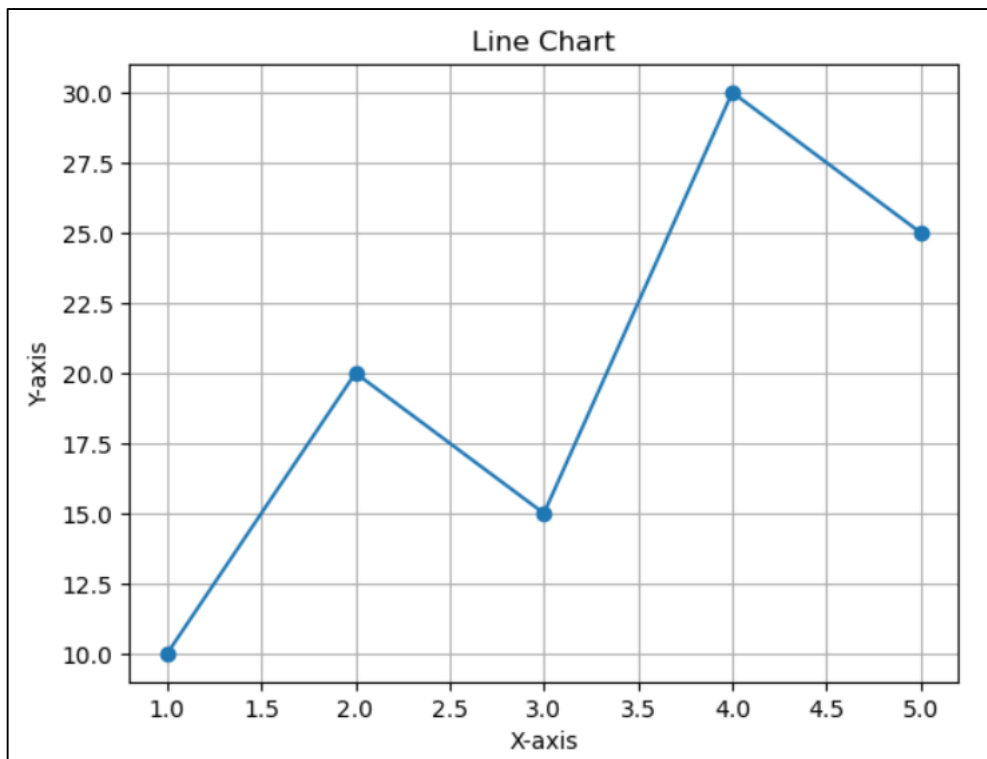
Output:

Practical 3: Develop Python programs to demonstrate various Data Visualization techniques.

a. Line Chart

```
import matplotlib.pyplot as plt  
  
x = [1, 2, 3, 4, 5]  
  
y = [10, 20, 15, 30, 25]  
  
plt.plot(x, y, marker='o')  
  
plt.title("Line Chart")  
  
plt.xlabel("X-axis")  
  
plt.ylabel("Y-axis")  
  
plt.grid(True)  
  
plt.show()
```

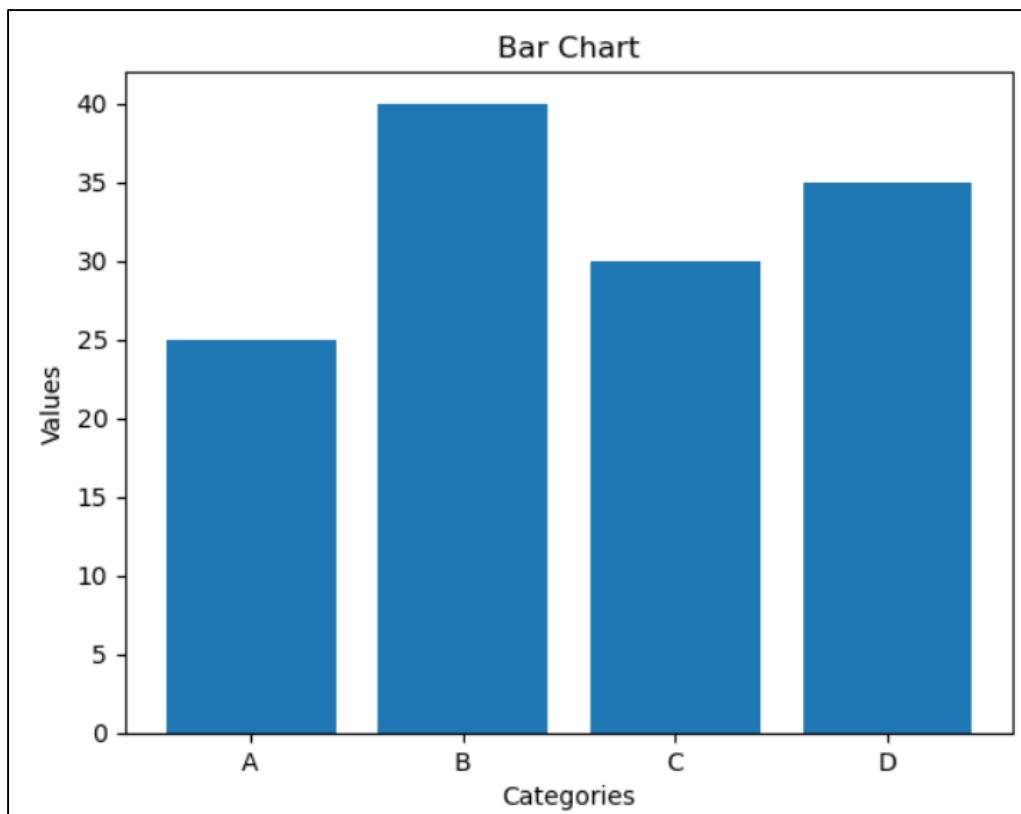
Output:



b. Bar Chart

```
import matplotlib.pyplot as plt  
categories = ['A', 'B', 'C', 'D']  
values = [25, 40, 30, 35]  
plt.bar(categories, values)  
plt.title("Bar Chart")  
plt.xlabel("Categories")  
plt.ylabel("Values")  
plt.show()
```

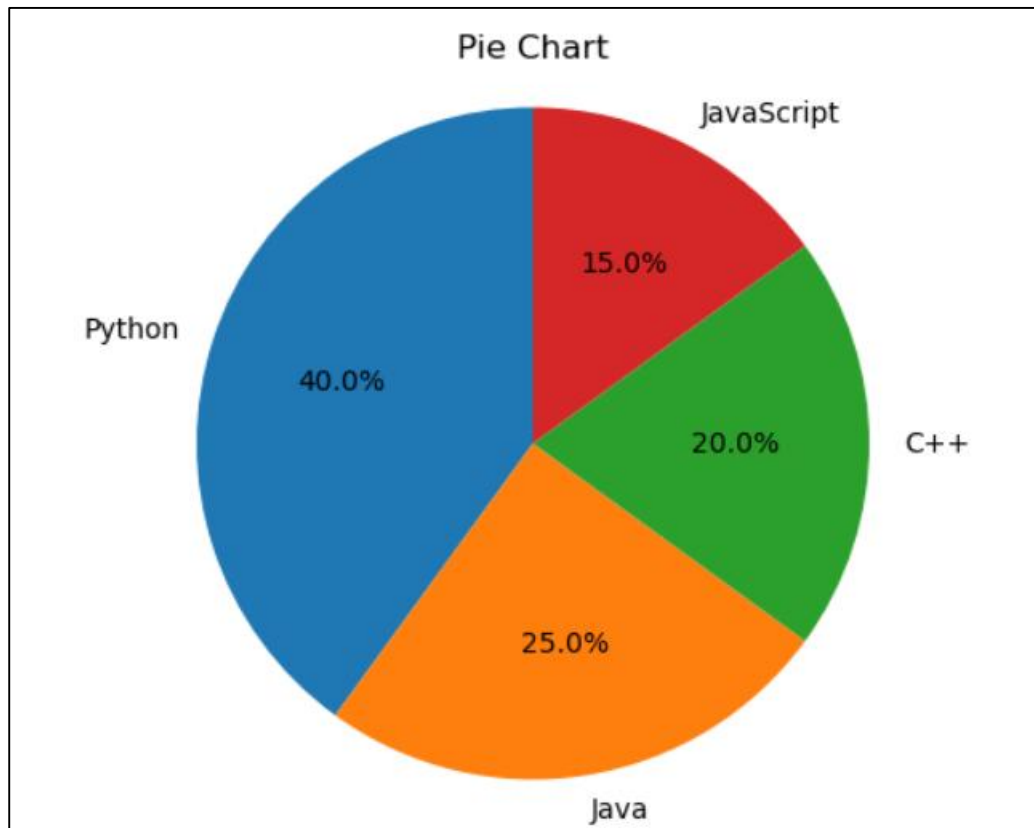
Output:



c. Pie Chart

```
import matplotlib.pyplot as plt  
labels = ['Python', 'Java', 'C++', 'JavaScript']  
sizes = [40, 25, 20, 15]  
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)  
plt.title("Pie Chart")  
plt.axis('equal')  
plt.show()
```

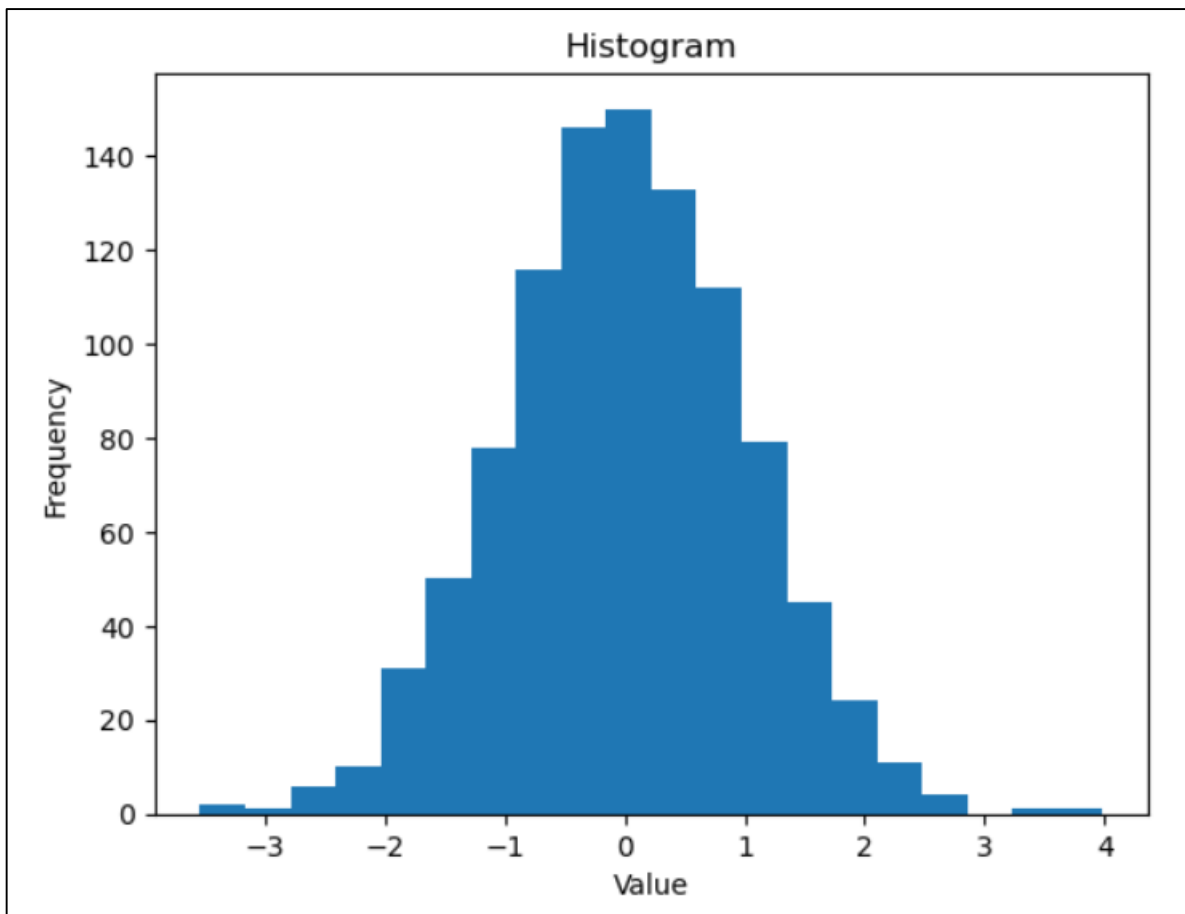
Output:



d. Histogram

```
import matplotlib.pyplot as plt  
  
import numpy as np  
  
data = np.random.randn(1000)  
  
plt.hist(data, bins=20)  
  
plt.title("Histogram")  
  
plt.xlabel("Value")  
  
plt.ylabel("Frequency")  
  
plt.show()
```

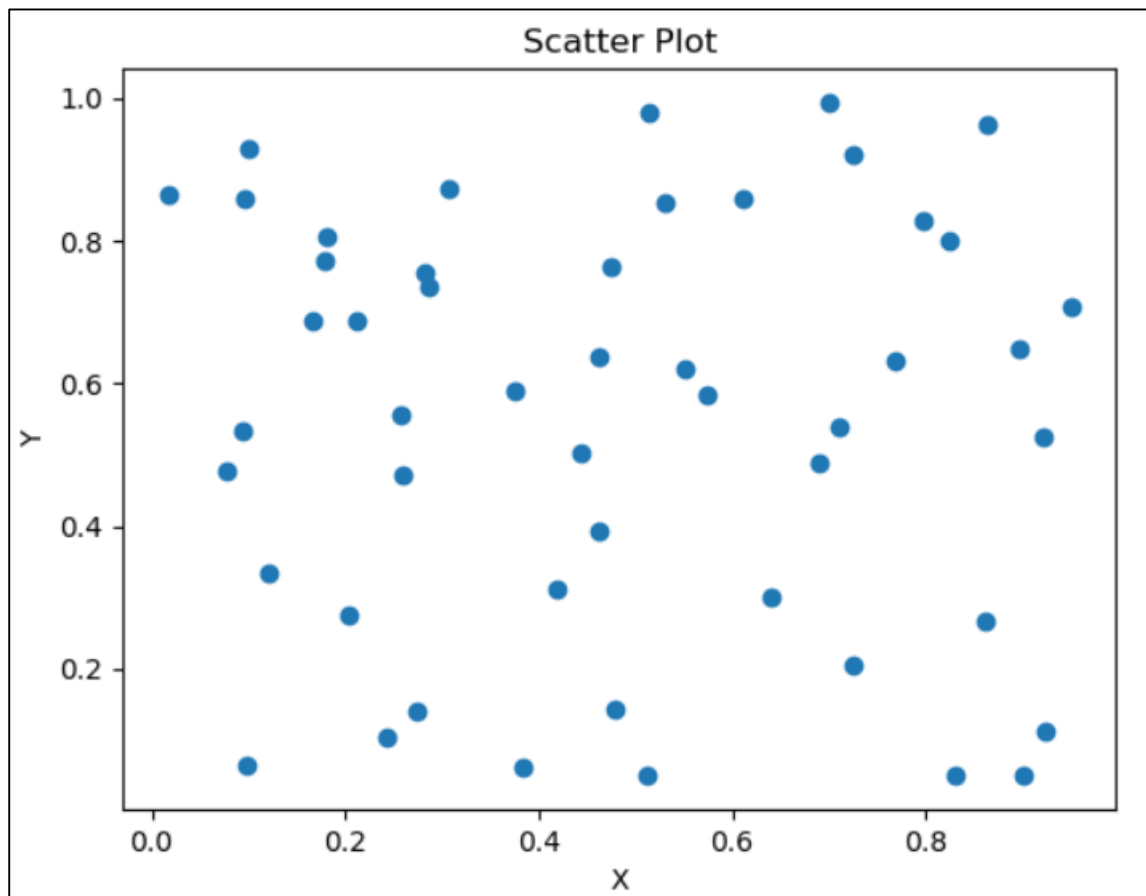
Output:



e. Scatter Plot

```
import matplotlib.pyplot as plt  
  
import numpy as np  
  
x = np.random.rand(50)  
y = np.random.rand(50)  
  
plt.scatter(x, y)  
  
plt.title("Scatter Plot")  
  
plt.xlabel("X")  
  
plt.ylabel("Y")  
  
plt.show()
```

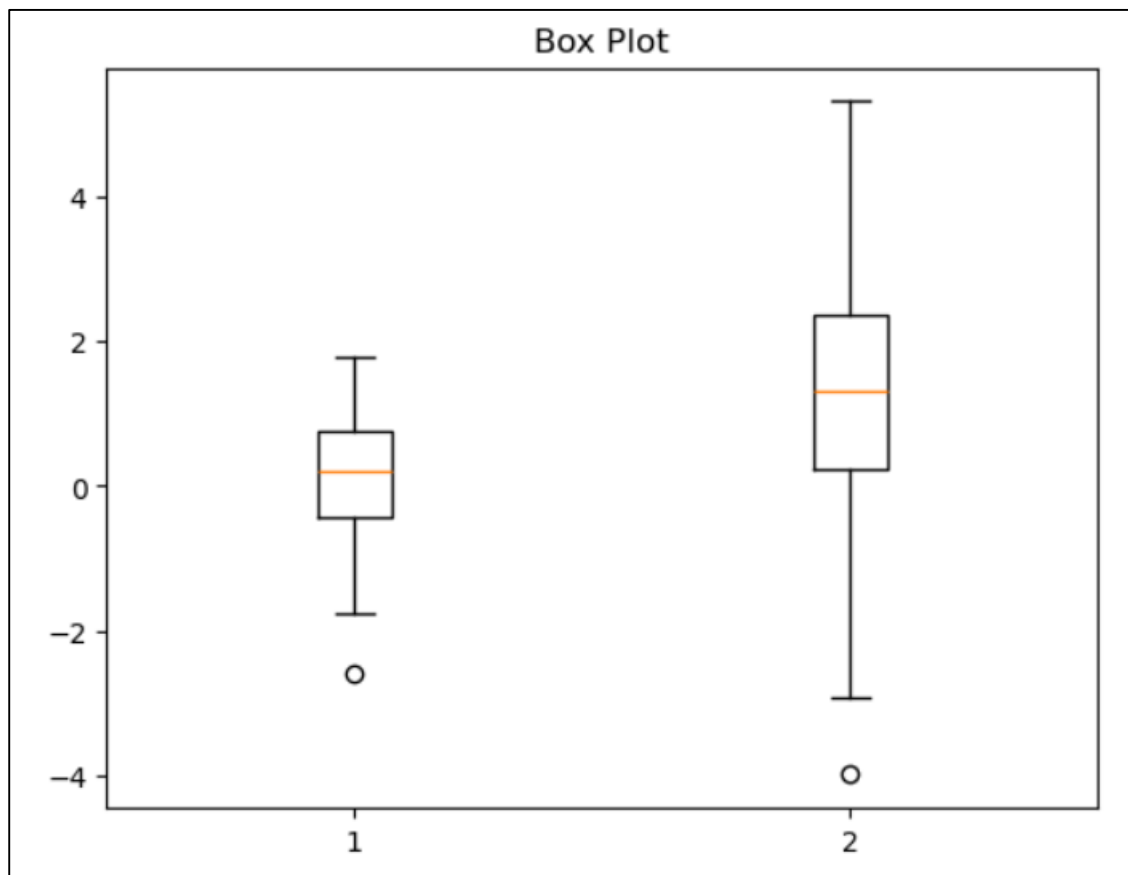
Output:



f. Box Plot

```
import matplotlib.pyplot as plt  
  
import numpy as np  
  
data = [np.random.normal(0, 1, 100), np.random.normal(1, 2, 100)]  
  
plt.boxplot(data)  
  
plt.title("Box Plot")  
  
plt.show()
```

Output:



Practical 4: Implement Linear Regression

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)

marks = np.array([35, 40, 50, 55, 60, 65, 70, 78, 84, 90])

model = LinearRegression()

model.fit(hours, marks)

print("Slope (m):", model.coef_[0])

print("Intercept (c):", model.intercept_)

new_hours = np.array([[6.5]])

predicted_marks = model.predict(new_hours)

print("Predicted marks for 6.5 hours of study:", predicted_marks[0])

plt.scatter(hours, marks, color="blue", label="Actual data")

plt.plot(hours, model.predict(hours), color="red", label="Regression line")

plt.xlabel("Hours Studied")

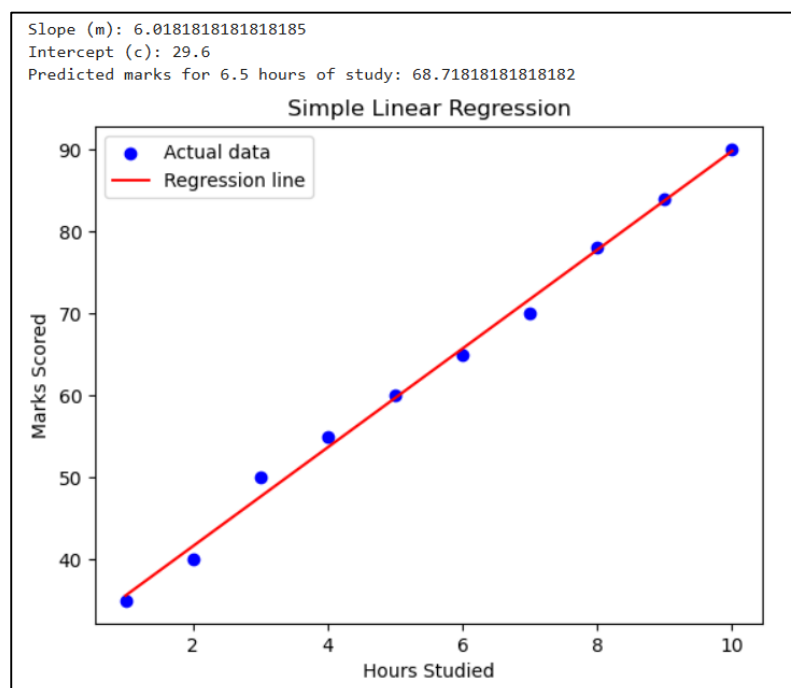
plt.ylabel("Marks Scored")

plt.title("Simple Linear Regression")

plt.legend()

plt.show()
```

Output:



Practical 5: Implement K-Nearest Neighbor Classifier

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

X = np.array([ [150, 7], [170, 7.5], [140, 6.5], [130, 6], [300, 10], [280, 9.5], [310, 10.5], [290, 9.8] ])

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

k=3

model = KNeighborsClassifier(n_neighbors=k)

model.fit(X, y)

new_fruit = [[160, 7]]

prediction = model.predict(new_fruit)

result = "Apple" if prediction[0] == 0 else "Orange"

print("The new fruit is predicted to be:", result)

plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color="green", label="Apple")

plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color="orange", label="Orange")

plt.scatter(new_fruit[0][0], new_fruit[0][1], color="red", marker="*", s=200, label="New Fruit")

plt.xlabel("Weight (grams)")

plt.ylabel("Size (cm)")

plt.title("KNN Classifier: Apple vs Orange")

plt.legend()

plt.show()
```

Output:

